Experiment 4: NeuroCore-32 RISC-V Processor

FPGA Build Challenge 2026 - Intermediate level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

RISC-V is an open instruction set that anyone may implement. The objective of this experiment is to design a 32-bit RISC-V processor for the RV32I base instruction set in the FPGA fabric, using the classic five-stage pipeline (fetch, decode, execute, memory, write back) so that up to five instructions are worked on at once. The pipeline is kept correct and fast by a forwarding unit, a hazard unit and a branch predictor, and hardware counters measure how well it performs. The ARM processor loads programs into the core over AXI4-Lite, starts it and reads the results back. Small programs - Fibonacci numbers, a bubble sort and an IoT sensor alarm - show it working on the board.

2. Block Diagram

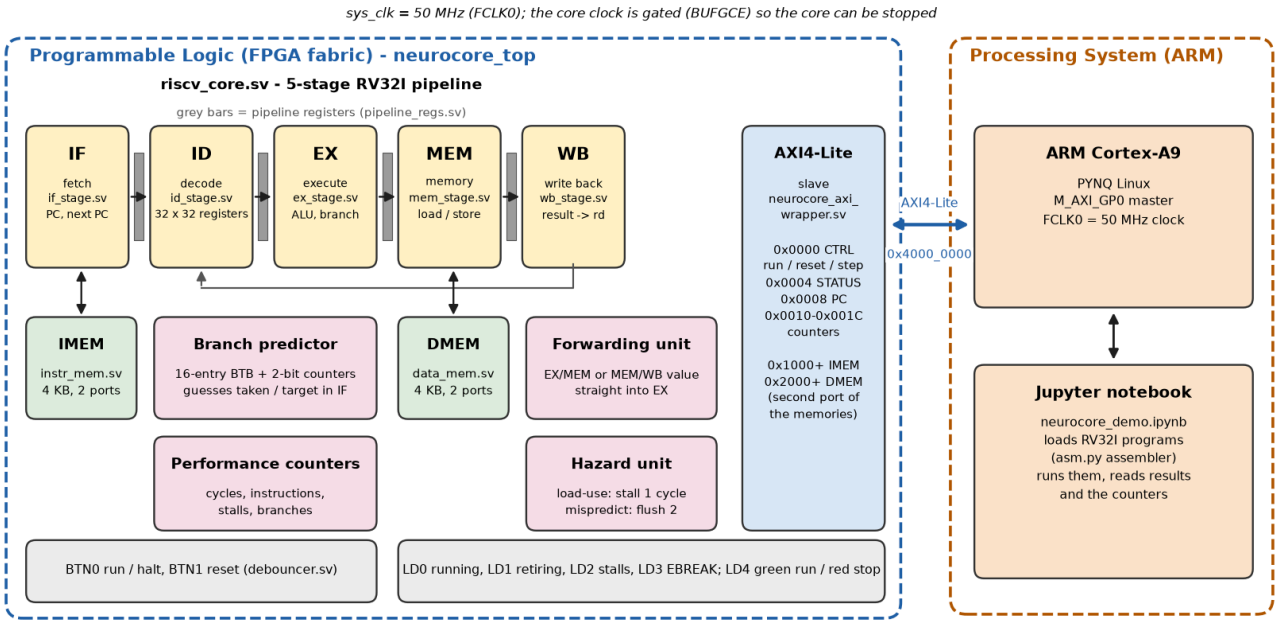


Figure 1 - System block diagram

The ARM writes a program into the 4 KB instruction memory and any input data into the 4 KB data memory through the second port of each memory, then sets the run bit. The core fetches, decodes and executes the program on its own until it reaches EBREAK, which stops the core clock and sets the halt flag. The ARM then reads the results from data memory and the performance counters (cycles, instructions retired, stalls, branches) from the status registers.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                  (Vivado block design, FCLK0 = 50 MHz)
 +-- neurocore_top_0            neurocore_top.sv  (buttons, LEDs, clock gate)
      |-- u_deb_btn0, u_deb_btn1  debouncer.sv
      |-- u_axi_wrapper          neurocore_axi_wrapper.sv
      +-- u_riscv_core           riscv_core.sv
           |-- if_stage          if_stage.sv + instr_mem.sv
           |-- id_stage          id_stage.sv (decoder, 32 x 32-bit register file)
           |-- ex_stage          ex_stage.sv (ALU, branch compare and target)
           |-- mem_stage         mem_stage.sv + data_mem.sv
           |-- wb_stage          wb_stage.sv
           |-- if_id / id_ex / ex_mem / mem_wb registers   pipeline_regs.sv
           |-- forwarding_unit   forwarding_unit.sv
           |-- hazard_unit       hazard_unit.sv
           |-- branch_predictor  branch_predictor.sv
           +-- perf_counters     perf_counters.sv
defines.sv                      opcodes, ALU operations and the pipeline register types
```

3.2 Modules

Module (file)	What it does
if_stage, instr_mem	Holds the PC and fetches the instruction. The next PC is PC + 4, the predicted target from the branch predictor, or the correct address after a misprediction. The instruction memory is 1024 x 32 bits with a second port for the ARM.
id_stage	Decodes the instruction (R, I, S, B, U and J formats of RV32I), builds the immediate, reads two registers from the 32 x 32-bit register file (x0 is always 0) and produces the control signals.
ex_stage	ALU (add, sub, and, or, xor, shifts, set-less-than), compares for branches and computes the branch or jump target. Its inputs can come from the forwarding paths.
mem_stage, data_mem	Loads and stores words, half-words and bytes in the 1024 x 32-bit data memory, which has a second port for the ARM.
wb_stage	Chooses the ALU result, the loaded value or PC + 4 (for jumps) and writes it to register rd.
pipeline_regs	The IF/ID, ID/EX, EX/MEM and MEM/WB registers, with stall (hold) and flush (clear) inputs.
forwarding_unit	If an instruction in EX needs a register that an older instruction in MEM or WB is about to write, the value is taken from that stage instead of the register file.
hazard_unit	Load-use hazard (a load followed by an instruction that needs the loaded value): stall one cycle. Branch mispredicted in EX: flush the two instructions fetched after it.
branch_predictor	16-entry branch target buffer with a 2-bit saturating counter per entry, looked up with the fetch PC. A hit with the counter in a "taken" state sends fetch straight to the stored target.
perf_counters	Counts clock cycles, retired instructions, stall cycles and branches.
neurocore_axi_wrapper	AXI4-Lite slave: control (run, reset, single step), status, PC, current instruction, the four counters and the two memory windows. A BUFGCE gates the core clock, so the core stops at EBREAK.
neurocore_top	Joins the core and the wrapper, debounces BTN0 (run / halt) and BTN1 (reset) and drives the LEDs.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x0000	CTRL	W	bit 0 run, bit 1 reset, bit 2 single step
0x0004	STATUS	R	bit 0 running, bit 1 stopped at EBREAK
0x0008 / 0x000C	PC / INSTR	R	program counter and current instruction
0x0010 - 0x001C	COUNTERS	R	cycles, instructions retired, stalls, branches
0x1000 +	IMEM	W	instruction memory (written while the core is stopped)
0x2000 +	DMEM	R/W	data memory

3.4 Test programs

The programs are written with a small Python assembler (Testbench/programs/asm.py) that encodes RV32I instructions and resolves labels, and are saved as .hex files.

Program	Size	What it does
fibonacci.hex	17 instructions	Computes the first 20 Fibonacci numbers and stores them at 0x100.
bubble_sort.hex	29 instructions	Writes 15 ... 0 into memory at 0x000 and sorts them with nested loops (loads, stores and many branches).
sensor_threshold.hex	23 instructions	Reads 16 temperatures from 0x000, writes an alarm flag (1 if the reading is 45 C or more) for each to 0x100 and the number of alarms to 0x180 - a simple IoT edge task.

3.5 RTL schematics

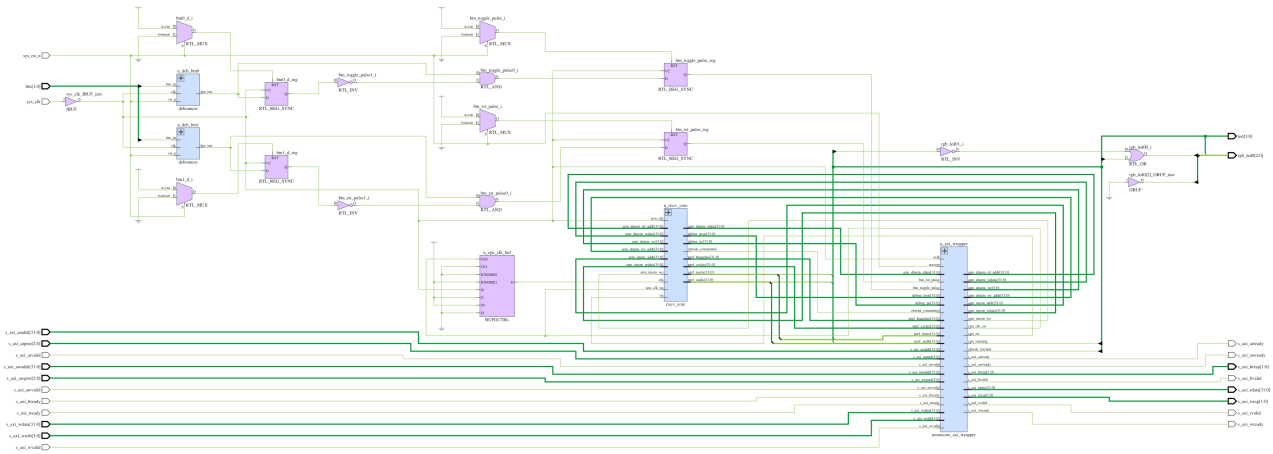


Figure 2 - Elaborated RTL schematic of neurocore_top (Vivado)

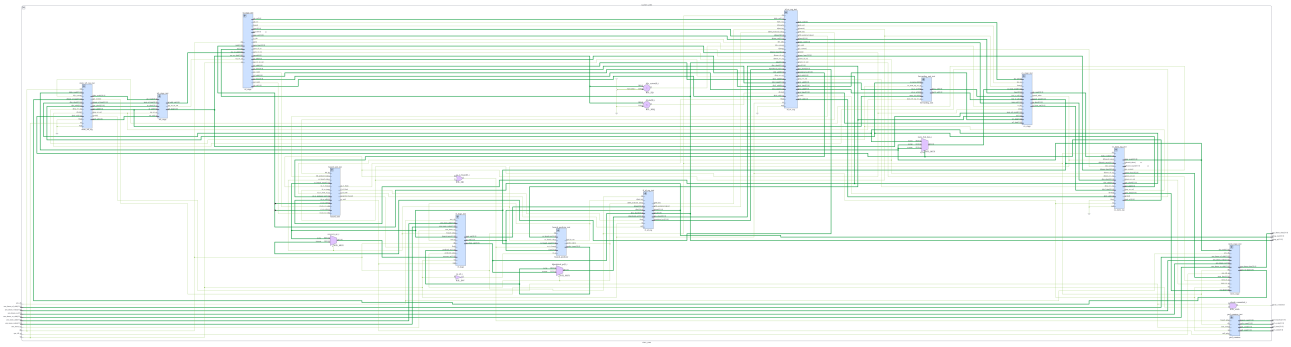


Figure 3 - Inside riscv_core: the five stages, pipeline registers, forwarding, hazard and branch-prediction units (full size in Images/rtl_schematic_core.png)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *neurocore_pynq_tb.sv*, which executes four verification phases: Fibonacci sequence generation, bubble sort, IoT sensor threshold detection, and an EBREAK regression test. All checks passed:

```
loaded fibonacci.hex (17 instructions)
program stopped at EBREAK (status 00000002)
[PASS] the first 20 Fibonacci numbers are correct
  cycles 179, instructions 171, stalls 0, branches 37
  IPC: 0.955
loaded bubble_sort.hex (29 instructions)
program stopped at EBREAK (status 00000002)
[PASS] the array was sorted into 0 to 15
loaded sensor_threshold.hex (23 instructions)
program stopped at EBREAK (status 00000002)
[PASS] sensor alarm flags are correct
loaded fibonacci.hex (17 instructions)
program stopped at EBREAK (status 00000002)
[PASS] EBREAK regression: F15 = 610 (post-sensor store was correctly suppressed)
PASS: NeuroCore-32 ran all programs correctly
```

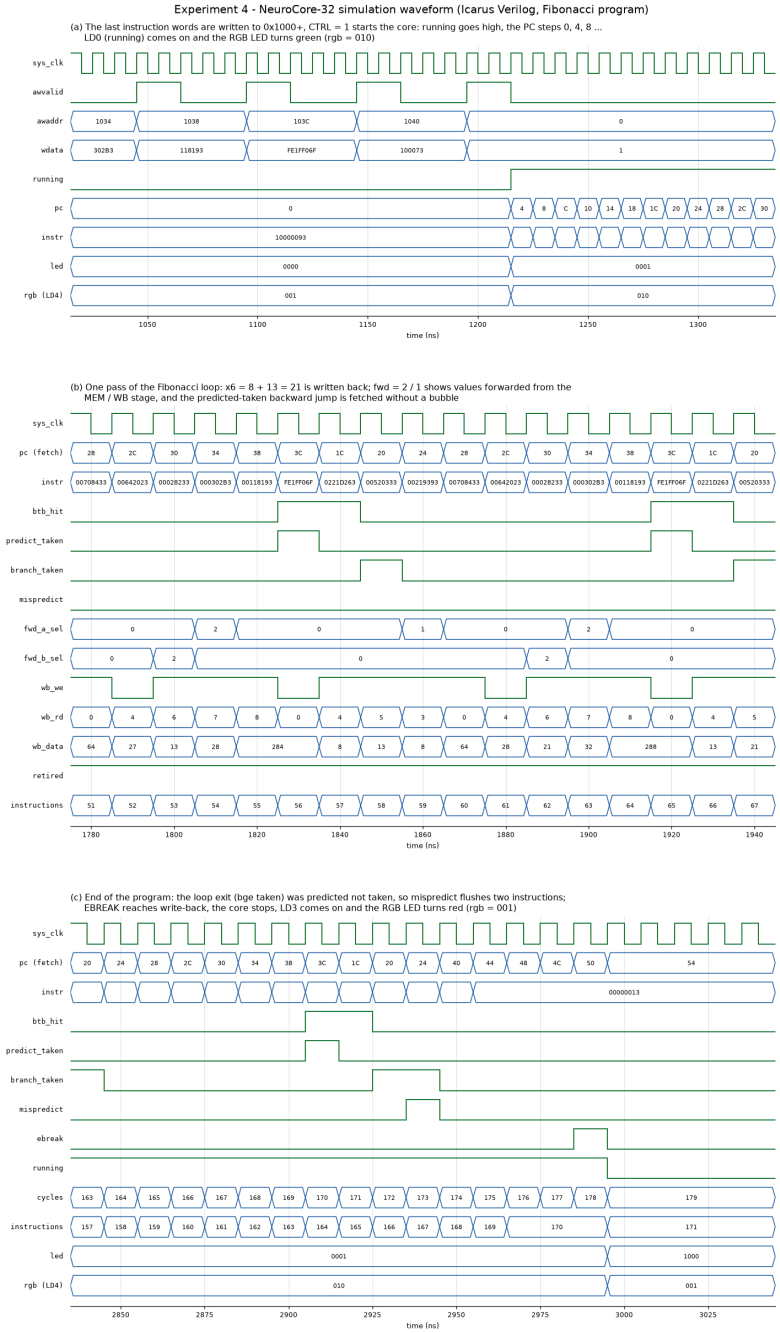


Figure 4 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl, with the fabric clock set to 50 MHz. The constraints file (pynq_z2.xdc) assigns the four LEDs, the RGB LED LD4 and the two buttons. The two 4 KB memories are built from LUTs (distributed RAM), which is most of the "LUT as memory" figure.

Resource	Used	Available	Utilisation
Slice LUTs	4,801	53,200	9.02 %
of which LUT as memory (IMEM, DMEM)	2,344	17,400	13.47 %
Slice registers (flip-flops)	2,347	106,400	2.21 %
Block RAM tiles	0	140	0.00 %
Bonded I/O pins	9	125	7.20 %
Global clock buffers (incl. the core clock gate)	2	32	6.25 %

Timing (50 MHz clock)	Value	Status
Worst setup slack (WNS)	+4.104 ns	**met**
Worst hold slack (WHS)	+0.022 ns	**met**
Timing endpoints	23,851	

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 and the three programs were run exactly as the notebook neurocore_demo.ipynb does it: reset the core, write the program (and, for the sensor program, 16 temperature readings) through the AXI memory windows, start it, wait for EBREAK and read the results and the counters.

Check	Detail	Result
Fibonacci: first 20 numbers	F19 = 4181	**PASS**
Fibonacci: counters equal simulation	179 cycles, 171 instructions (sim: 179, 171)	**PASS**
Bubble sort: array is 0 to 15	0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	**PASS**
Sensor: alarm flags	0011010101100101	**PASS**
Sensor: anomaly count	8 readings >= 45 C	**PASS**
All programs stopped at EBREAK	STATUS = 0x00000002	**PASS**

Program	Cycles	Instructions	Stalls	Branches	IPC	Time at 50 MHz
Fibonacci	179	171	0	37	0.955	3.58 us
Bubble sort	1575	1411	120	439	0.896	31.50 us
Sensor threshold	224	186	0	57	0.830	4.48 us

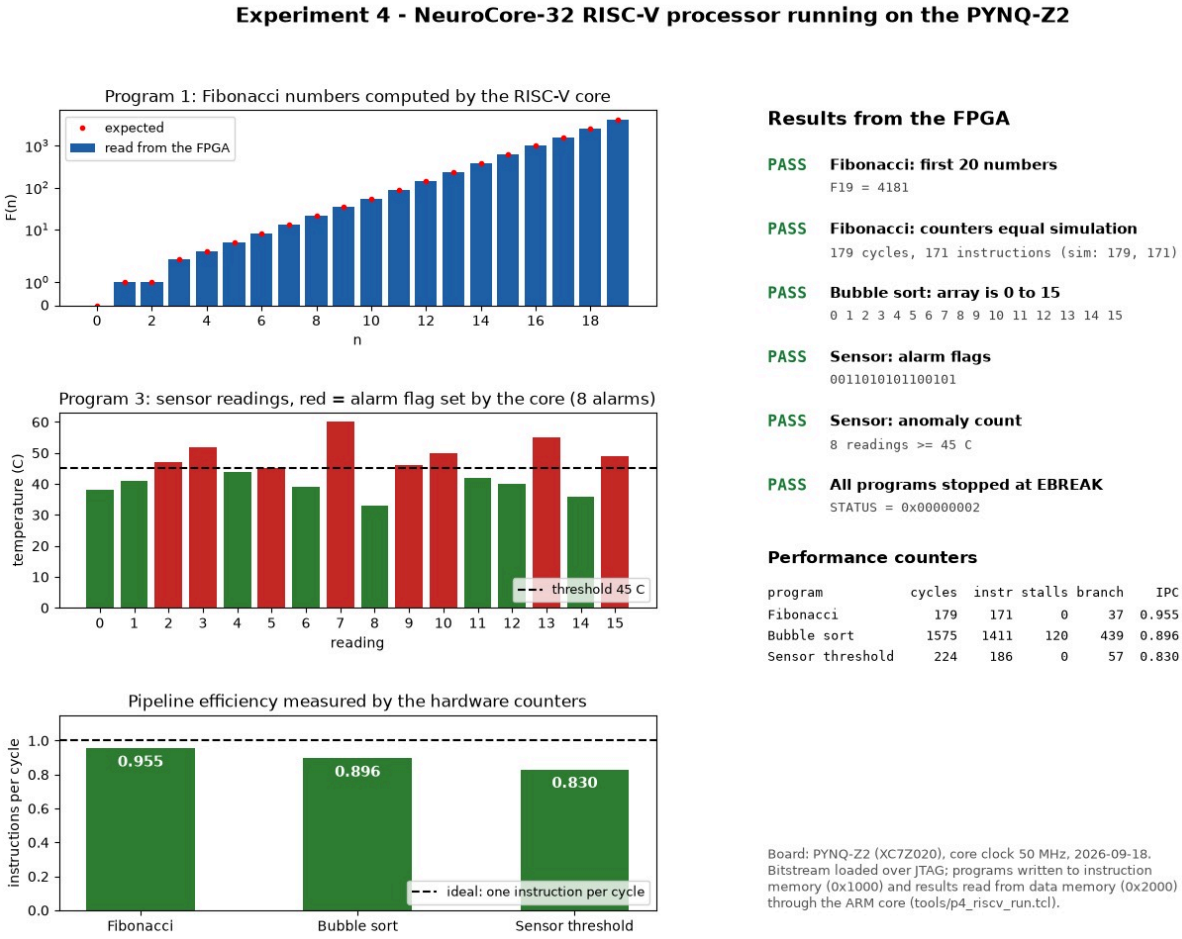


Figure 5 - Programs run by the RISC-V core on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook neurocore_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It ran the Fibonacci and bubble sort programs. Both results are correct, and the counters are exactly the same as in the run above and in simulation: 171 instructions in 179 cycles (IPC 0.955) for Fibonacci, and 1411 instructions in 1575 cycles with 120 stalls and 439 branches for the sort. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 4 (Neurocore RISC-V)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, processor at 0x40000000
Fibonacci numbers from the FPGA:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
correct: PASS
171 instructions in 179 cycles = 0.955 IPC
array after sorting: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
correct: PASS
1411 instructions in 1575 cycles, 120 stalls, 439 branches
```

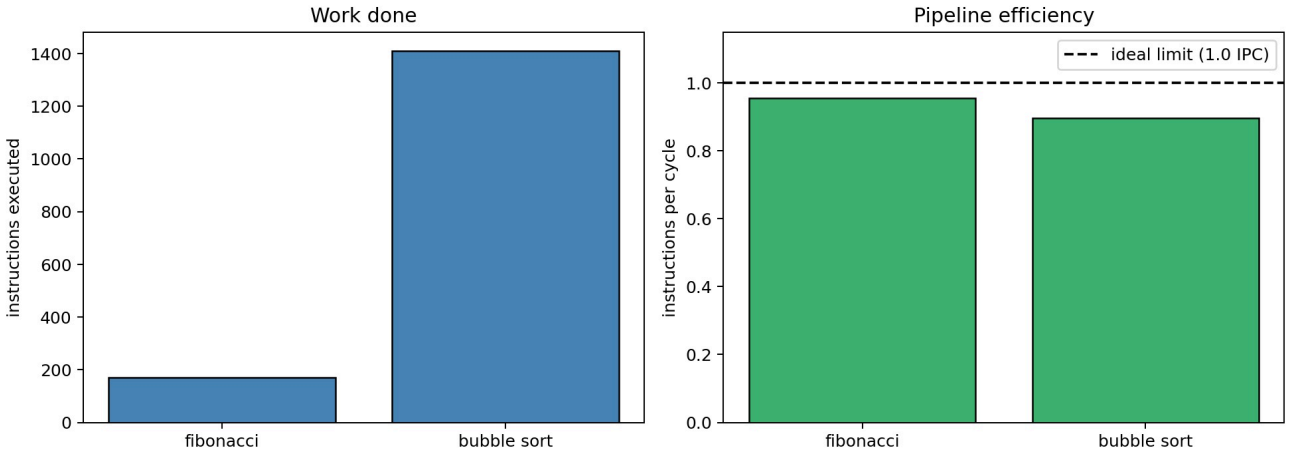


Figure 6 - Graph drawn by neurocore_demo.ipynb on the board

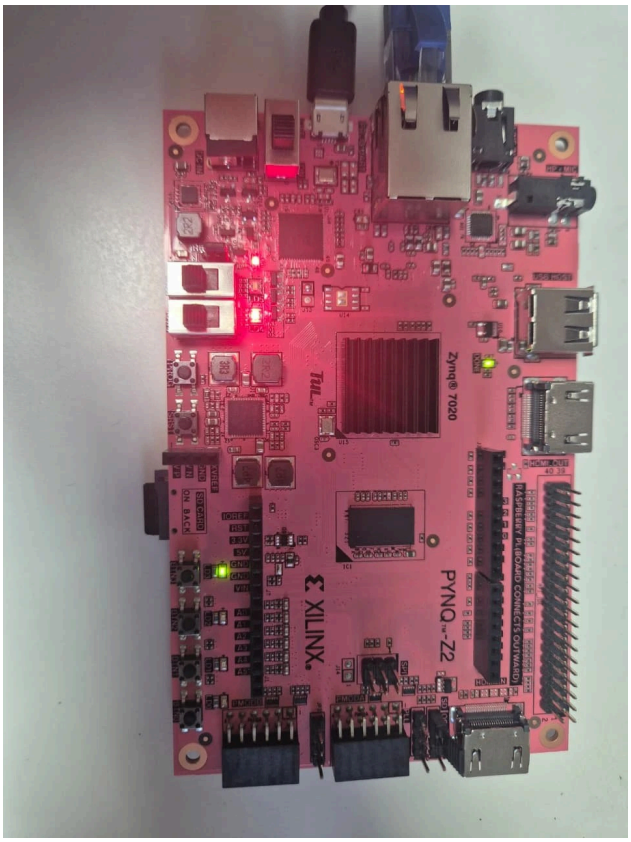


Figure 7 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

All three programs produced the correct results, and the Fibonacci program took exactly the same number of cycles on the board as in simulation, as expected for a synchronous design. The pipeline completes close to one instruction per clock. The few lost cycles come from filling the pipeline at the start, from the two instructions flushed whenever a branch is mispredicted (for example the last iteration of each loop), and in the bubble sort from load-use stalls, where an instruction needs the value that the previous load has not yet returned.

5.3 A bug found on the board: a store after EBREAK

While testing on the board, the Fibonacci program was run straight after the sensor program. F15 (data address 0x13C) then read back as 1 instead of 610. The cause: loading Fibonacci (17 words) overwrites only the start of the instruction memory, so the word after its EBREAK was still an instruction left over from the sensor program, *sw x11, 0(x10)* (0x00B52023). The core fetches past EBREAK, so this store was one stage behind EBREAK in the pipeline. Fibonacci does not use x10 and x11, and the register file

is not cleared by a reset, so they still held the sensor program's last values: x10 = 0x13C (the address of its last alarm flag) and x11 = 1. The store wrote 1 to 0x13C in the same clock that EBREAK reached write-back, just before the core clock stopped.

Fix (riscv_core.sv): the data memory write enable is blocked while EBREAK is in the write-back stage.

```
wire ebreak_in_wb = mem_wb_out.valid & mem_wb_out.is_system;
...
.mem_wr_en (ex_mem_out.mem_wr_en & ~ebreak_in_wb),
```

A fourth test was added to the testbench that repeats this sequence (sensor program, then Fibonacci) and checks that F15 is 610. It fails with the old RTL (F15 = 1) and passes with the fix (section 4). The board results above were taken with the bitstream built before the fix; in the order used there (Fibonacci, sort, sensor) they were not affected. The bitstream was then rebuilt with the fix (2026-09-19, timing met, numbers in section 5.1); the fix itself is checked in simulation, and the new bitstream has not yet been run on the board.

6. Applications

- **Embedded and IoT controllers:** a small soft processor next to custom hardware, running control code such as the sensor alarm program on the edge instead of sending raw data away.
- **Customisable processors:** because RISC-V is open, new instructions or accelerators can be added for one application.
- **Teaching and research:** the counters make pipeline effects (stalls, mispredictions) visible and measurable.
- **Management cores:** a small CPU inside a larger FPGA design to handle configuration and housekeeping.

7. Conclusion

A five-stage pipelined RV32I processor was designed, simulated and implemented on the PYNQ-Z2. It meets timing at 50 MHz, runs programs loaded by the ARM processor, and reached an IPC close to 1 on the board with correct results for all three test programs. Testing on the board also found a bug, a store that could still complete after EBREAK; it was fixed in the RTL and a regression test was added (section 5.3).

Learning outcomes: the RV32I instruction formats and how to decode them; building a pipeline with pipeline registers; data hazards and how forwarding and stalling solve them; control hazards and how a branch target buffer with 2-bit counters reduces their cost; measuring performance with hardware counters; and sharing memories between a soft processor and the ARM over AXI.

Future scope: add the M extension (multiply and divide); use block RAM for larger memories; add interrupts and a UART so the core can talk on its own; add a misprediction counter to measure the predictor's accuracy directly; and run C programs compiled with the RISC-V GCC toolchain.